# TP : Authentification des utilisateurs dans une application Spring MVC avec variables de session

## Table of Contents


1. Objectifs du TP . . . 1
2. Préparer la gestion des utilisateurs . . . 1
   2.1. Objectif . . . 1
   2.2. Étapes . . . 2
   2.3. Questions . . . 4
3. Implémenter la logique de connexion . . . 4
   3.1. Objectif . . . 5
   3.2. Travail à faire . . . 8
4. Centraliser la vérification du laisser-passer . . . 8
   4.1. Objectif . . . 8
   4.2. Étapes . . . 8
5. Ajouter la fonctionnalité de déconnexion . . . 8
   5.1. Objectif . . . 8
   5.2. Étapes . . . 9
6. Fin du TP . . . 9


# 1. Objectifs du TP

Nous allons intégrer un formulaire de connexion à l'application et introduire la notion d'utilisateur. À partir de maintenant, toutes les fonctionnalités seront réservées à des personnes authentifiées. L'authentification sera gérée à l'aide d'une variable de session.

> Il existe des solutions plus avancées pour gérer l'authentification, comme Spring Security, mais elles ne sont pas abordées dans ce cours.

# 2. Préparer la gestion des utilisateurs

## 2.1. Objectif

Mettre en place les fondations nécessaires à l'authentification des utilisateurs dans l'application.

## 2.2. Étapes

1. Créez une classe `Utilisateur` dans le package `modele` :

```
public class Utilisateur {
    private String email;
    private String nom;
    private String prenom;
    private String motDePasse;

    public Utilisateur(String email, String nom, String prenom, String motDePasse) {
        this.email = email;
        this.nom = nom;
        this.prenom = prenom;
        this.motDePasse = motDePasse;
    }
    // Getters et Setters
}
```

1. Créez un DTO nommé `AuthentificationDTO` dans le package `dto` :

```
public class AuthentificationDTO {
    @NotNull(message="L'email ne peut pas être vide")
    @NotBlank(message="L'email ne peut pas être vide")
    private String email;

    @NotNull(message="Le mot de passe ne peut pas être vide")
    @NotBlank(message="Le mot de passe ne peut pas être vide")
    private String motDePasse;

    public AuthentificationDTO() {}
    public AuthentificationDTO(String email, String motDePasse) {
        this.email = email;
        this.motDePasse = motDePasse;
    }

    // Getters et Setters
}
```

1. Ajoutez une classe `LaisserPasser` :

```
public class LaisserPasser {
    private String email;
    private String cleSession;

    public LaisserPasser(String email) {
        this.email = email;
        this.cleSession = UUID.randomUUID().toString();
```

```
    }

    public String getEmail() {
        return email;
    }

    public String getCleSession() {
        return cleSession;
    }
}
```

1. Modifiez la classe `GestionEvenements` :

```
@Component
public class GestionEvenements {
    private int compteur = 0;
    private List<Evenement> evenements = new ArrayList<>();
    private Map<String,Utilisateur> utilisateurs = new HashMap<>();
    private Collection<LaisserPasser> laisserPassers = new ArrayList<>();

    public GestionEvenements() {
        utilisateurs.put("alice@gmail.com", new Utilisateur("alice@gmail.com",
"Classique", "Alice", "alice"));
        utilisateurs.put("bob@gmail.com", new Utilisateur("bob@gmail.com", "Classique
", "Bob", "bob"));
    }

    public LaisserPasser authentifier(AuthentificationDTO dto) throws
UtilisateurInconnuException {
        Utilisateur utilisateur = utilisateurs.get(dto.getEmail());
        if (utilisateur != null && utilisateur.getMotDePasse().equals(dto
.getMotDePasse())) {
            LaisserPasser laisserPasser = new LaisserPasser(utilisateur.getEmail());
            laisserPassers.add(laisserPasser);
            return laisserPasser;
        }
        throw new UtilisateurInconnuException("Utilisateur inconnu ou mot de passe
incorrect");
    }

    public Utilisateur getUtilisateurByEmail(String email) {
        return utilisateurs.get(email);
    }

    public int enregistrer(LaisserPasser laisserPasser, EvenementDTO dto) throws
LaisserPasserInvalideException {
        if (!laisserPassers.contains(laisserPasser)) throw new
LaisserPasserInvalideException();
        Evenement evenement = new Evenement(compteur, dto.getNom(), dto.getLieu(),
dto.getDate());
```

```
        compteur++;
        dto.setIdentifiant(evenement.getIdentifiant());
        evenements.add(evenement);
        return evenement.getIdentifiant();
    }

    public int enregistrer(LaisserPasser laisserPasser, String nom, String lieu,
String date) throws LaisserPasserInvalideException {
        if (!laisserPassers.contains(laisserPasser)) throw new
LaisserPasserInvalideException();
        Evenement e = new Evenement(compteur, nom, lieu, date);
        compteur++;
        evenements.add(e);
        return e.getIdentifiant();
    }

    public Evenement trouverParId(LaisserPasser laisserPasser, int id) throws
EvenementInconnuException, LaisserPasserInvalideException {
        if (!laisserPassers.contains(laisserPasser)) throw new
LaisserPasserInvalideException();
        return evenements.stream()
            .filter(e -> e.getIdentifiant() == id)
            .findFirst()
            .orElseThrow(() -> new EvenementInconnuException("Événement inconnu : " +
id));
    }

    public List<Evenement> lister(LaisserPasser laisserPasser) throws
LaisserPasserInvalideException {
        if (!laisserPassers.contains(laisserPasser)) throw new
LaisserPasserInvalideException();
        return evenements;
    }

    public void logout(LaisserPasser laisserPasser) {
        laisserPassers.remove(laisserPasser);
    }
}
```

## 2.3. Questions

- Explorez les nouvelles fonctionnalités de la façade.
- Quelle différence notable observez-vous sur les méthodes déjà existantes ? Quelle est l'utilité de cette modification ?

# 3. Implémenter la logique de connexion

## 3.1. Objectif

Compléter le contrôleur `EvenementControleur` pour gérer le formulaire de connexion et créer la page de connexion.

```
@Controller
@RequestMapping("/mesevenements")
public class EvenementControleur {
    private final GestionEvenements gestion;
    public EvenementControleur(GestionEvenements gestion) {
        this.gestion = gestion;
    }

    @GetMapping("/")
    public String index() {
        return "redirect:/mesevenements/loginform";
    }

    @GetMapping("/creationform")
    public String gotoCreationForm(Model model) {
        model.addAttribute("evenementDTO", new EvenementDTO());
        return "creation";
    }

    @GetMapping("/loginform")
    public String gotoLoginForm(Model model) {
        //TODO
        return "login";
    }


    @GetMapping("/home")
    public String gotoHome(Model model, HttpSession session, RedirectAttributes
redirectAttributes) {
        //TODO
        return "home";
    }

    @PostMapping("/login")
    public String authentifier(
            @Valid @ModelAttribute AuthentificationDTO authentificationDTO,
            BindingResult bindingResult,
            HttpSession session,
            RedirectAttributes redirectAttributes,
            Model model
    ) {
        //TODO
    }
```

```
    @PostMapping("/creation")
    public String ajouterViaModel(@Valid@ModelAttribute EvenementDTO evenementDTO,
BindingResult bindingResult, Model model,HttpSession session, RedirectAttributes
redirectAttributes) {

        if (bindingResult.hasErrors()) {
            return "creation";
        }

        if (session.getAttribute("laisserPasser") == null) {
            redirectAttributes.addFlashAttribute("erreur", "Vous devez vous connecter
pour ajouter un événement.");
            return "redirect:/mesevenements/loginform";
        }
        LaisserPasser laisserPasser = (LaisserPasser) session.getAttribute(
"laisserPasser");
        int identifiant = 0;
        try {
            identifiant = gestion.enregistrer(laisserPasser,evenementDTO);
            model.addAttribute("message", "Événement ajouté avec succès !");
            model.addAttribute("identifiant", identifiant);
            return "resume";
        } catch (LaisserPasserInvalideException e) {
            redirectAttributes.addFlashAttribute("erreur", "Laisser-passer non valide,
veuillez vous reconnecter.");
            return "redirect:/mesevenements/loginform";
        }

    }


    @GetMapping("/evenement/{id}")
    public String afficherUn(@PathVariable int id, Model model, RedirectAttributes
redirectAttributes, HttpSession session) {
        if (session.getAttribute("laisserPasser") == null) {
            redirectAttributes.addFlashAttribute("erreur", "Vous devez vous connecter
pour ajouter un événement.");
            return "redirect:/mesevenements/loginform";
        }
        LaisserPasser laisserPasser = (LaisserPasser) session.getAttribute(
"laisserPasser");
        try {

            Evenement evenement = gestion.trouverParId(laisserPasser,id);
            EvenementDTO evenementDTO = new EvenementDTO();
            evenementDTO.setIdentifiant(evenement.getIdentifiant());
            evenementDTO.setNom(evenement.getNom());
            evenementDTO.setLieu(evenement.getLieu());
```

```
            evenementDTO.setDate(evenement.getDate());
            model.addAttribute("evenement", evenementDTO);
            model.addAttribute("id", id);
            return "detail";
        } catch (EvenementInconnuException e) {
            model.addAttribute("erreur", "Événement introuvable.");
            return "erreur";
        } catch (LaisserPasserInvalideException e) {
            redirectAttributes.addFlashAttribute("erreur", "Laisser-passer non valide,
veuillez vous reconnecter.");
            return "redirect:/mesevenements/loginform";
        }
    }


    @GetMapping("/evenements")
    public String lister(Model model, HttpSession session, RedirectAttributes
redirectAttributes) {
        if (session.getAttribute("laisserPasser") == null) {
            redirectAttributes.addFlashAttribute("erreur", "Vous devez vous connecter
pour ajouter un événement.");
            return "redirect:/mesevenements/loginform";
        }
        LaisserPasser laisserPasser = (LaisserPasser) session.getAttribute(
"laisserPasser");
        try {
            model.addAttribute("evenements", gestion.lister(laisserPasser));
            return "liste";
        } catch (LaisserPasserInvalideException e) {
            redirectAttributes.addFlashAttribute("erreur", "Laisser-passer non valide,
veuillez vous reconnecter.");
            return "redirect:/mesevenements/loginform";
        }

    }

        @RequestMapping("/logout")
        public String logout(HttpSession session) {
            LaisserPasser laisserPasser = (LaisserPasser) session.getAttribute(
"laisserPasser");
            gestion.logout(laisserPasser);
            session.invalidate(); // Invalidate the session to clear all attributes
            return "redirect:/";
        }
}
```

Une route a changé de nom entre temps `/mesevenements/creer` est devenu `/mesevenements/creation`. Pensez à mettre à jour votre formulaire dans `creation.html`.

## 3.2. Travail à faire

1. Créez la page `login.html` avec affichage des erreurs, liée au DTO `AuthentificationDTO` permettant à un utilisateur de se connecter en saisissant son email et son mot de passe.
2. Implémentez la méthode `gotoLoginForm`.
3. Implémentez la méthode `authentifier` pour :
   - Appeler la méthode `authentifier` de la façade
   - Stocker le laisser-passer en session
   - Rediriger vers `home.html` avec un message de bienvenue
   - Gérer les erreurs et redirections
4. Créez la page `home.html` avec :
   - Un message de bienvenue personnalisé
   - Un lien vers `/mesevenements/creationform`
   - Un lien vers `/mesevenements/evenements`

# 4. Centraliser la vérification du laisser-passer

## 4.1. Objectif

Supprimer la redondance de la vérification du `laisserPasser` dans chaque méthode du contrôleur et la redirection induite.

## 4.2. Étapes

1. Créez un `HandlerInterceptor` :
   - Excluez les chemins `/`, `/mesevenements/login` et `/mesevenements/loginform` de cette vérification.
   - Vérifie si la session contient `laisserPasser`
   - Si absent, redirige vers `/mesevenements/loginform`
2. Enregistrez l'intercepteur dans une classe qui implémente `WebMvcConfigurer`.

# 5. Ajouter la fonctionnalité de déconnexion

## 5.1. Objectif

Permettre à l'utilisateur de se déconnecter proprement.

## 5.2. Étapes

1. Ajoutez un lien de déconnexion dans `home.html` pointant vers `/mesevenements/logout`
2. Vérifiez que la méthode `logout()` du contrôleur fonctionne :
    - Supprime le laisser-passer de la façade
    - Invalide la session
    - Redirige vers `/`

# 6. Fin du TP

Vous avez maintenant une application avec une gestion d'authentification simple basée sur les sessions HTTP. Bravo !